What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)
Affiliation
Address
email

Abstract

Coding agents are evaluated on the patch they leave behind. We show that this final state hides most of what happens during a run. We built an instrument that records every mutating action of an agent as a git commit on a hidden reference, replays each benchmark instance's held-out tests at every recorded state inside the benchmark's own container image, and attributes each detected regression to the harness check that could have caught it. Applied to SWE-bench Verified, the instrument recorded 184 regression events across the runs that produced any; the final patch exposed one. Regression frequency depended on the model stack: on the same 40 instances under the same harness, one frontier stack produced no events in 40 runs and another produced 140 events in 11 incidents (Fisher p = 0.010). A pre-registered comparison of recovery policies, analysed with code committed before unblinding, found that rollback to a verified checkpoint left contamination in one final tree where no recovery left it in nine (paired sign test, p = 0.022), but rollback also resolved fewer tasks, because the verifier that triggers it rejected patches the official grader accepted. Replacing that verifier with the repository's own tests removed the tradeoff: 65% resolved with no contaminated final tree (McNemar p = 0.012 against plain rollback). We release the instrument, its validation protocol, and a catalogue of 28 measurement defects encountered while building it, 23 of which produced plausible numbers rather than errors.

1 Introduction

Benchmarks for coding agents grade the final patch. SWE-bench [1] and its Verified subset [2] run the repository's tests against the patch an agent submits and report whether the target tests now pass and the previously passing tests still pass. Work on regressions introduced by agents follows the same convention: it examines the submitted patch and counts the previously passing tests it breaks [8].

This convention has a blind spot. A regression that an agent introduces and later repairs leaves no trace in the final patch, and any harness with a recovery mechanism produces that pattern by design. Whether it matters depends on two unanswered empirical questions: how often regressions occur during a run, and how much of that activity survives to the final state.

We answer both by measuring the timeline rather than the endpoint. Our contributions are:

1. **An instrument for event-level regression measurement.** Every mutating tool call is recorded as a commit on a git reference the agent cannot see; after the run, the instance's held-out passing tests are replayed at every recorded state inside the benchmark's pinned container, and detection is attributed by coverage. Validity is established by negative and

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

positive controls, a liveness gate, a golden check that drives the gold patch through the agent's execution path, and a re-scoring control that re-measures archived runs with no model calls (Section 3).

2. **Measurements on SWE-bench Verified.** Across the GPT-5.6 rollback runs that produced any regression, the timeline recorded 184 events and the final state exposed one (Section 5.2). Event frequency depended on the model stack: identical instances and harness gave 0 events under one frontier stack and 140 under another (Section 5.3).
3. **A pre-registered comparison of recovery policies, and a fix.** With the analysis committed before unblinding, rollback to a verified checkpoint left fewer contaminated final trees than no recovery ($p = 0.022$) but resolved fewer tasks; we trace the loss to verifier precision, and a post hoc arm with a regression-gated verifier built from the instrument's own oracle removes it, reaching 65% resolve with no contaminated tree (Sections 5.4 to 5.6).
4. **A catalogue of 28 measurement defects** found while building the instrument, classified by mechanism, with the design rules that caught most of them (Appendix A); 23 produced a plausible number rather than an error.

All measurements are exploratory and were made on a development slice of the benchmark that is excluded from any confirmatory frame. No claim is made about the population of instances beyond that slice.

2 Background and related work

Benchmarks and their validity. SWE-bench [1] grades a patch by tests that must go from failing to passing and tests that must keep passing; Verified [2] is the human-screened 500-instance subset. Concerns about the static subset have accumulated: contamination, memorisation, and leaked solutions [6], flawed tests [7], and rolling [3, 5] or private [4] alternatives in response. UTBoost [7] showed the held-out tests are often insufficient, changing 24% of Verified leaderboard verdicts. TDAD [8] measured regressions in submitted patches on Verified and found them in about 30% of resolved instances; we measure the same quantity on the timeline, and Section 5.2 compares the two on the same runs.

Agent scaffolds and recovery. SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent [12] span the scaffold design space, on the action formalisms of CodeAct [13] and ReAct [14]. Within-episode recovery has been studied as self-reflection [15, 16], progress-gated recovery [17], checkpoint repair [18], and provenance-based rollback [19]; operating-system framings [20] treat checkpointing as a shared primitive. Process reward models and rubric verifiers [24, 25] approach the verifier-precision problem of Section 5.5 from the training side.

Automated program repair. The regression problem is the plausible-versus-correct patch problem of program repair [26, 28, 29]; regression tests are the main defence against overfitting patches [27]. Our results concern regressions the agent's own process repairs before the patch is tested; trajectory-level evaluation [21–23] argues more generally that resolve rate alone is uninformative. Public leaderboard trajectories record actions and observations but not the tree at each step, so the measurement here cannot be mined from them without re-executing the trajectory; the instrument commits the tree.

3 Instrument

Figure 1 shows the harness and the instrument. **Observational timeline.** The harness wraps every mutating tool call. After each call, the working tree is committed to a git reference outside the agent's view, using a private index so that the agent's own `git status` and `git diff` are unchanged. Rollbacks and the end of the run are recorded as observations too. Because observations are commits, an archived run can be re-measured later without re-running the agent.

Exhaustive replay. For each observation, the instance's passing-test set is run inside the pinned container against that observation's tree; a regression event is a test that passes at one observation and fails at a later one. Every observation is replayed rather than bisected, because bisection assumes monotone verdicts and a recovery policy violates that by construction. Replay is scoped to the files holding the held-out tests, which is what keeps it affordable: 4 to 8 seconds per observation on the

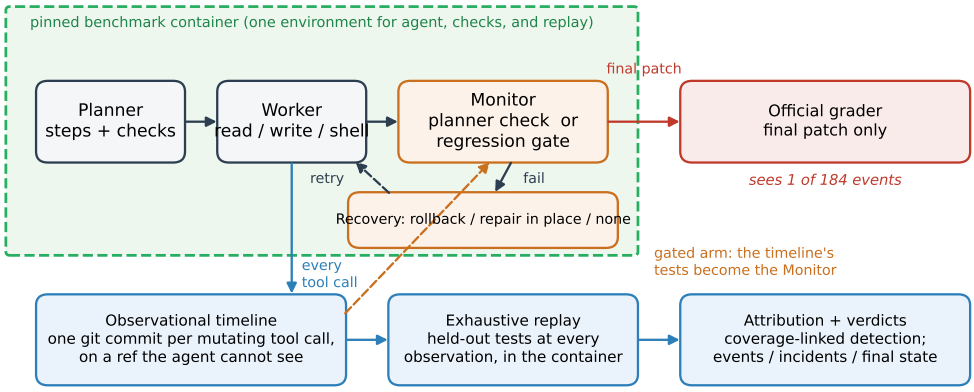

Figure 1: The harness and the instrument. The planner, worker, monitor, and recovery policy run inside the benchmark's pinned container; every mutating tool call is committed to a hidden git reference; held-out tests are replayed at every observation and detections are attributed by coverage. In the gated arm (dashed), the timeline's tests serve as the monitor. The official grader sees only the final patch.

two repository families we timed (pytest and django), about 26 CPU-minutes for a 40-instance sweep, and a projected 80 CPU-hours for 500 instances at 100 edits each. Infrastructure failures during replay are recorded as missing observations rather than as test failures, and baseline-dead tests are excluded from event counting.

Attribution. A detected regression is classified three ways: *attributed* if a failing harness check and the broken held-out test both exercise a file the agent changed at that observation (using a per-instance coverage map built at the base commit), *co-occurring* if some harness check failed while the regression was open but no such link exists (an over-count of detection), and *unknown* if the coverage map cannot say.

Execution environment. The agent's tools and the harness's checks execute inside the same pinned container as the replay, with file changes synchronised to the host tree the timeline records (Appendix A.2 describes what it cost to learn this).

Validation. Five gates run before any paid experiment: a negative control, a positive control with injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and a baseline liveness check that the probes return passes. A golden check drives the benchmark's gold patch through the agent's real tool path and requires the grader to return "resolved" (and a null run "unresolved"); it passed on three repository families with distinct runners. A re-scoring control re-measures an archived run's committed timeline under a changed instrument with no model calls; on both runs to date it reproduced the archived episode counts.

4 Experimental setup

Benchmark and slice. SWE-bench Verified, 500 instances. A 40-instance development slice (16 from django), stratified and fixed before any measurement, was used throughout and is excluded from any confirmatory frame. The GPT-5.6 stack was also run on the slice's first 10 instances as a calibration, and plain rollback was run twice; pooled denominators are stated where used.

Harness. A planner decomposes the task into steps with verification commands; a worker executes each step with three tools (read file, write file, run shell); a monitor runs the verification, and on failure the recovery policy acts: **rollback** (reset to the last verified checkpoint, retry with feedback), **repair-in-place** (retry from the failed tree), or **no recovery** (keep the failed step's tree). Each run has a $4 work-cost cap; exhaustion is scored as failure.

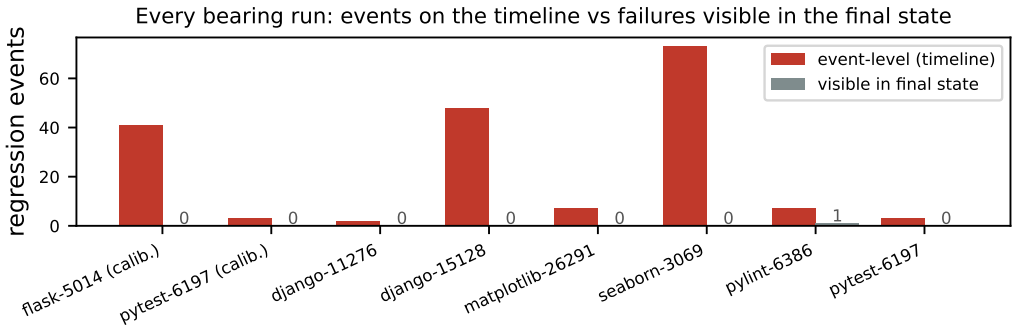

Figure 2: Every run that produced at least one regression event, in both GPT-5.6 sweeps. Left bars (red) are events recorded on the timeline; right bars (grey) are held-out test failures visible in the graded final patch.

Models. Two frontier stacks under identical harness, policy, instances, and caps, named by their API model strings: `claude-opus-4-7` (planner) with `claude-sonnet-4-6` (worker), and `gpt-5.6-sol` (planner) with `gpt-5.6-terra` (worker); identifiers are recorded in every run manifest.

Outcomes. Resolve is the official grader's verdict on the final patch. Because the event distribution is heavily overdispersed, events are reported at three levels: distinct incidents (observations at which at least one test broke), declared events (one per test function and onset, parametrised variants collapsed), and bearing runs. Final-state contamination is the number of held-out tests failing in the graded final patch, net of baseline-dead tests.

Pre-declaration. The event unit, the gates, and the contrast were declared before the relevant data existed; the contrast's analysis (exact paired sign tests on final-state contamination as primary and incident exposure as co-primary) was committed before either comparison arm finished. One amendment, making the gates conditional on the model stack, preceded unblinding. Appendix B gives the wording.

5 Results

5.1 Resolve and cost

Under rollback, the Claude stack resolved 13 of 40 instances (32.5%, exact 95% CI 18.6% to 49.1%) at $34.24; the GPT-5.6 stack resolved 13 of 40 (32.5%; 13 of the 35 graded, 37.1%) at $8.14. Three GPT-5.6 cells were never attempted (the sweep's circuit breaker stopped after six consecutive zero-progress failures in one repository family) and two were lost to a file-synchronisation defect; all five count as unresolved out of 40 and are excluded from the graded denominator of 35. Paired comparisons below use only instances graded in both arms.

5.2 The final state hides almost all regression activity

Figure 2 shows every run that produced a regression event under the GPT-5.6 stack with the rollback policy, pooling the 10-instance calibration and the 40-instance sweep (47 runs). The timeline recorded 184 declared events across eight runs; the final patch exposed one. Run-weighted, seven of the eight bearing runs ended with a clean final patch; three storms (73, 48, and 41 events) supply 88% of the event count, so the run-weighted figure is the more robust statement. Measured from the final patch, as prior work does [8], the same runs show one event.

Figure 3 shows a single run in detail. The grader marked it resolved with all graded tests passing; its timeline contains three regressions of the same test function at observations 3, 5, and 7, each repaired by rollback. Two of the three went undetected by any harness check while open; the repairing rollback was triggered by an unrelated failure. Across the full GPT-5.6 sweep, 48 of 162 raw episodes (events before collapsing parametrised variants) had no co-occurring harness failure at all.

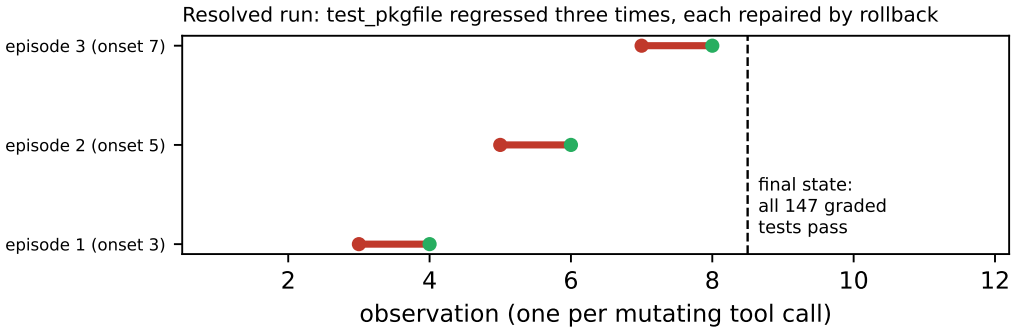

Figure 3: One run that the official grader marked resolved, with all 147 graded tests passing. The timeline shows three regressions of the same test function, each repaired by rollback two observations later.

5.3 Regression frequency depends on the model stack, not the benchmark

On the same 40 instances, under the same harness and rollback policy, the Claude stack produced no regression events in 40 runs (227 observations, all replayed). The GPT-5.6 stack produced 140 declared events in 11 incidents across 6 of 37 attempted runs (bearing fraction 16.2%, exact CI 6.2% to 32.0%). A one-sided Fisher exact test on the bearing fractions gives p = 0.010; this is a single pairwise test on six bearing runs, and the two stacks also traverse different provider adapters. Observation density differs by a factor of 1.2 (5.7 and 7.0 per run), which does not account for 140 to 0. The two stacks resolved the same number of instances (13 of 40 each) at $34.24 against $8.14: the stack that never broke adjacent behaviour cost four times as much per run. The three storms that supply 88% of the events are model edits, not harness artifacts: a six-line parseable change to seaborn's `plot.py`, a nine-line change to django's `query.py`, and a 262-line deletion in flask's `blueprints.py`, none truncated and none containing a shell idiom; the output-cap defect (D10) was fixed before these sweeps, and their event logs contain no capped turns.

The Claude result is a measured zero rather than a detection failure: the same stack's earlier canary run produced three events, and re-scoring its archived timeline under the instrument used for the 40-run sweep reproduces them.

We had first read the low event rate as a property of the benchmark, and a pre-declared rule directed a switch to one with a larger held-out set; the second stack falsified that on the same instances. The pre-declared gate (event rate at least 0.30 per run and bearing fraction at least 25%) nevertheless failed for both stacks: the GPT-5.6 bearing fraction was 20% at calibration and 16.2% at full scale. No arm in this paper is a confirmatory pass, and the amendment that made the gates conditional on the model stack was a response to that failure. In our single-sweep comparison, regression frequency depended on the model stack (0 of 40 versus 6 of 37); we do not claim more than that.

5.4 Recovery policy: rollback keeps the final tree clean

Figure 4 summarises the pre-declared contrast. The primary endpoint, final-state contamination, is paired by instance: no recovery was worse than rollback on 9 instances, better on 1, and tied on 25 (exact sign test, p = 0.022). Repair-in-place was worse on 5, better on 1, and tied on 29 (p = 0.22). The co-primary endpoint, incident exposure, did not differ detectably for either comparison (no recovery p = 0.45; repair-in-place p = 1.0): regressions occurred at similar rates under every policy, and the policy determined whether they persisted.

One no-recovery run illustrates the mechanism: its final tree begins with the literal string `$(cat django/db/models/expressions.py)`, a shell idiom pasted into a file write, and none of the instance's 137 graded tests ran; under rollback the same model's failed attempt was reset and the final patch passed all 137.

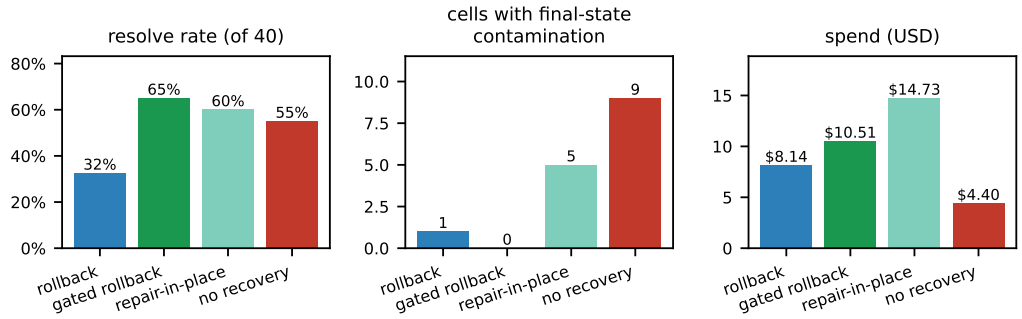

Figure 4: The four recovery arms under the GPT-5.6 stack on the same 40 instances. Contamination counts cells whose final patch fails a previously-passing test.

Disclosure. The first unblinding gave p = 0.375; Appendix B records the grading correction (seven catastrophically contaminated cells had been dropped as ungradable) and the re-grade of their archived trees.

5.5 Rollback loses resolution to verifier precision

Rollback resolved fewer tasks than either alternative (Figure 4, left): 13 of 40 against 24 of 40 for repair-in-place and 22 of 40 for no recovery; paired by instance, it won one discordant pair and lost nine against repair-in-place (McNemar exact p = 0.022) and won three and lost nine against no recovery (p = 0.15). The loss is attributable to the verifier: of the 18 rollback runs that failed, 15 failed at the first step, and 9 of the 18 were resolved under a policy that kept the rejected work. The monitor's planner-written check had rejected a patch the grader accepted, and rollback discarded it, so the verifier's false-rejection rate sets how many correct patches are lost per contaminated tree avoided.

5.6 Regression-gated rollback removes the tradeoff

A fourth arm, added after the contrast was unblinded and therefore exploratory, replaces the monitor's planner-written check with the repository's previously-passing tests, run in the agent's container at the start of the run and after every attempt; a step is rejected only if a test that passed at the start fails now. It resolved **26 of 40 instances (65.0%)**, the most of any arm, with **no contaminated final tree** (Figure 4). Paired against plain rollback on the 35 shared instances, it resolved 10 that plain rollback did not and lost 1 (McNemar exact p = 0.012; post hoc, one of seven tests reported, unadjusted). Paired onset exposure did not differ detectably (p = 0.73). The gate runs the test files holding the instance's previously-passing tests, selected from benchmark metadata, and its baseline oracle coincides with the grading oracle (median ratio 1.00), so the clean final trees are guaranteed by construction; the informative result is the resolve gain, and a split variant graded on test files the gate never sees is the next experiment. A second full sweep of plain rollback reproduced the first (13 and 13 resolved on 34 paired instances; 6 of 37 bearing runs both times), so the gain is well outside run-to-run variability.

6 Discussion

Why timeline regressions matter. A regression the agent repairs before submission did not reach the user, so the objection that it is harmless deserves an answer. It is not free: across the bearing runs, 27% of spend ($0.63 of $2.33) went to work done while a regression was open. The events are also the per-step ground truth that process reward models for software agents [24, 25] need, and the transient states include destructive ones (a tree with every test uncollectable) that no final-state grade records. For agent-OS design, a recovery primitive without an observation primitive hides exactly the activity that distinguishes recovery policies.

On the measurement defects. Building the instrument produced 28 defects (Appendix A), 23 of which produced a plausible number rather than an error. Two rules would have prevented most of them: record infrastructure failures as missing observations, and require a positive liveness signal.

Summary. Final-state evaluation misses the regressions an agent's own process repairs, which in our measurements is nearly all of them; measuring the timeline is feasible, validates against controls, separates recovery policies the final patch cannot, and locates rollback's cost in verifier precision.

7 Limitations

All measurements come from a 40-instance development slice: one sweep per arm, plus a calibration and one replication of plain rollback for the GPT-5.6 stack, so variability is measured for that arm only. The cross-stack comparison is a single pairwise test on six bearing runs, confounded with the provider adapter; the gated arm is post hoc. The held-out tests observe roughly the gold test patch's blast radius, so event counts are lower bounds; confidence intervals ignore repository clustering (16 of 40 instances are django); and the instrument has not been applied to a public scaffold, which is the next experiment.

References

[1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.

[2] OpenAI. Introducing SWE-bench Verified. 2024.

[3] L. Zhang et al. SWE-bench Goes Live! arXiv:2505.23419, 2025.

[4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.

[5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. arXiv:2505.20411, 2025.

[6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.

[7] H. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.

[8] TDAD: Regressions introduced by coding agents on SWE-bench Verified. arXiv:2603.17973, 2026.

[9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.

[10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.

[11] C. Xia et al. Agentless: Demystifying LLM-based software engineering agents. arXiv:2407.01489, 2024.

[12] SWE-agent team. mini-swe-agent. Software, 2025.

[13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.

[14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.

[15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.

[16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.

[17] ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.

[18] REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.

[19] From agent traces to trust: Evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2026.

[20] K. Mei et al. AIOS: LLM agent operating system. arXiv:2403.16971, 2024.

[21] Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977, 2025.

[22] What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184, 2026.

[23] SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, 2025.

[24] Agentic rubrics as contextual verifiers for SWE agents. arXiv:2601.04171, 2026.

[25] SWE-Shepherd: Advancing PRMs for reinforcing code agents. arXiv:2604.10493, 2026.

[26] Patch overfitting in program repair: A survey. 2024.

[27] Y. Wang et al. When automated program repair meets regression testing: An extensive study on two million patches. ACM TOSEM, 2024. arXiv:2105.07311.

[28] Patch correctness assessment: A survey. ACM TOSEM, 2024. doi:10.1145/3702972.

[29] H. Ye et al. Automated patch assessment for program repair at scale. Empirical Software Engineering, 2021.

[30] SWE-bench issue #601: Test result hijacking via stdout forging in the evaluation harness. github.com/SWE-bench/SWE-bench, 2025.

[31] OpenHands issues #4235 and #7044: Environment errors in SWE-bench instance evaluation. github.com/OpenHands/OpenHands.

[32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. arXiv:2504.21798, 2025.

[33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym. arXiv:2412.21139, 2024.

[34] Context as a tool: Context management for long-horizon SWE agents. arXiv:2512.22087, 2025.

A The measurement defect catalogue

A.1 The catalogue by mechanism

Each row is a defect found while building the instrument. **S**: it produced a plausible number rather than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only on a clean host. Class A rows depend on the machine the code runs on; class B rows render an infrastructure failure as a measurement; class C rows measure a state where an event was defined; class D rows consume a producer's output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare `python`	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare `pytest` from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare `python`	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family's oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to 2.7×	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured episodes reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host. Four further defects found after this census closed are described where they arose (Section 5.4 and the repository log).

A.2 The defect that invalidated a conclusion

After nineteen fixes, every validation gate passed, the re-scoring control reproduced archived episode counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared gate. We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source checkout, while every probe and every gate ran inside the pinned container. On the host, `import matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace package, and everything downstream fails in ways indistinguishable from an incompetent agent. Twenty-six of 28 zero-step runs traced to this. The gates had validated the measurement path and never the agent's execution path. The fix was to route the agent's tools and checks through the same container as the replay, and to add a per-cell environment parity check that runs before any model call.

A.3 Mechanisms in public infrastructure

Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-bench images. Class B: UTBoost's finding that the held-out oracle is insufficient at leaderboard scale [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8]. Class D: the SWE-bench grader accepting forged test output on stdout [30].

B Pre-declaration and analysis

The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction of at least 25%. Both stacks failed the conjunction (bearing 0% and 16.2%); the gates were then re-declared per (substrate, model stack) before the contrast was unblinded, and no arm reported here is confirmatory. The contrast's primary endpoint is final-state contamination, paired by instance, exact two-sided sign test; the co-primary is incident exposure. The analysis script was committed before either comparison arm finished. The regression-gated arm was added after the contrast was unblinded and is exploratory. No multiplicity adjustment was planned or applied. Budget exhaustion is scored as failure. Instances whose baseline oracle records failures in the unmodified image are excluded from resolve-rate comparability claims, and their dead tests are typed as missing observations for event counting.

Grading correction between unblindings. The first unblinding of the recovery-policy contrast gave $p = 0.375$ for the primary endpoint: seven cells (five no-recovery, two repair-in-place) had been dropped as ungradable because their final trees made the suite uncollectable. Official grading scores such a patch as failing every test, so the most contaminated states had been dropped from the endpoint they bore on most. The grader now distinguishes an environment failure from a patch that kills the suite, and the seven archived trees were re-graded without re-running any agent.